ISO/IEC JTC1 SC22 WG21 P0767R0
Jens Maurer <Jens.Maurer@gmx.net>
Target audience: LEWG, LWG and CWG
2017-09-29

P0767R0: Expunge POD

Introduction

This paper addresses NB commment US 101 on the CD for C++17. It is tracked as core issue 2323, but has substantial impact on the library section, too. In order to better handle the necessary coordination between LWG and CWG, the changes are presented in this paper.

Preliminary discussion on the reflector has demonstrated some concerns about the direction proposed in this paper. Notwithstanding, I believe the remaining use (but not the definition, if required for the library section) of POD in the core language section is misleading at best and should be fixed.

Wording changes

Change footnote in 6.8 [basic.life] paragraph 6:

> Before the lifetime of an object has started but after the storage which the object will occupy has been allocated [Footnote: For example, before the construction of a global object ~~of non-POD class type~~ **that is initialized via a user-provided constructor** (15.7 [class.cdtor]).] or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any pointer that represents the address of the storage location where the object will be or was located may be used but only in limited ways. [...]

Remove in 6.9 [basic.types] paragraph 9:

> [...] ~~Scalar types, POD classes (Clause 12), arrays of such types and cv-qualified versions of these types are collectively called POD types.~~ [...]

Change in 8.1.5.1 [expr.prim.lambda.closure] paragraph 2:

> An implementation may define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:
>
> - the size and/or alignment of the closure type,
> - whether the closure type is trivially copyable (Clause 12), **or**
> - whether the closure type is a standard-layout class (Clause 12)~~, or~~**.**
> - ~~whether the closure type is a POD class (Clause 12).~~

Remove the normative text in 12 [class] paragraph 10, but keep the example:

> ~~A POD struct [Footnote: ...] is a non-union class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD struct, non-POD union (or array of such types). Similarly, a POD union is a union that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD struct, non-POD union (or array of such types). A POD class is a class that is either a POD struct or a POD union.~~ [Example:

```
struct N {            // neither trivial nor standard-layout
  int i;
  int j;
  virtual ~N();
};
struct T {            // trivial but not standard-layout
  int i;
private:
  int j;
};
struct SL {           // standard-layout but not trivial
  int i;
  int j;
  ~SL();
};
struct POD {          // both trivial and standard-layout
  int i;
  int j;
};
```

-- end example]

Change in 20.3.3 [defns.character.container]:

character container type
a class or a type used to represent a character [Note: It is used for one of the template parameters of the string, iostream, and regular expression class templates. ~~A character container type is a POD (6.9) type.~~ -- end note]

Change in 21.2.4 [support.types.layout] paragraph 5:

The type `max_align_t` is a ~~POD~~ **trivial** type whose alignment requirement is at least as great as that of every scalartype, and whose alignment requirement is supported in every context (6.11 [basic.align]).

Change in 23.15.2 [meta.type.synop]

```
~~template <class T> struct is_pod;~~
[...]
~~template <class T> inline constexpr bool is_pod_v~~
~~    = is_pod<T>::value;~~
```

In the table in 23.15.4.3 [meta.unary.prop], strike the row for `is_pod`:

~~template <class T> struct is_pod;~~
~~T is a POD type (6.9 [basic.types])~~
~~`remove_all_extents_t<T>` shall be a complete type or cv void.~~

Change in the table in 23.15.7.6 [meta.trans.other]:

`aligned_storage`
The member typedef type shall be a ~~POD~~ **trivial** type suitable for use as uninitialized storage for any object whose size is at most `Len` and whose alignment is a divisor of `Align`.

`aligned_union`
The member typedef type shall be a ~~POD~~ **trivial** type suitable for use as uninitialized storage for any object whose type is listed in `Types`; its size shall be at least `Len`.

Change in 24.1 [strings.general] paragraph 1:

> This Clause describes components for manipulating sequences of any non-array ~~POD~~ **trivial** (6.9 [basic.types]) type. Such types are called char-like types, and objects of char-like types are called char-like objects or simply characters.

Add a new subsection to Annex D or merge into D.12 [depr.meta.types]:

> **D.17 Identification of POD types [depr.is_pod]**
>
> **The following type property is defined in header `<type_traits>` in addition those defined in 23.15.4.3 [meta.unary.prop]:**
>
> ```
> template <class T> struct is_pod;
> template <class T> inline constexpr bool is_pod_v
> = is_pod<T>::value;
> ```
>
> ***Requires:*** **`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.**
>
> **`is_pod<T>` is a UnaryTypeTrait (23.15.1 [meta.rqmts]) with a base characteristic of `true_type` if T is a POD type, and `false_type` otherwise. A POD class is a class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD class (or array thereof). A POD type is a scalar type, a POD class, an array of such a type, or a cv-qualified version of one of these types. [Note: It is unspecified whether a closure type (8.1.5.1 [expr.prim.lambda.closure]) is a POD type. -- end note]**